

Kudde - User & Admin Manual

Kudde ("herd", in Afrikaans) is a small, self-hosted app for small-scale cattle recording. It tracks the animals in a herd and the handful of things that happen to them - a birth, a weighing, a treatment, a move between camps, a sale, a death - from one server on the farm, which any phone, tablet or PC signed into the farm's Tailscale network can open from anywhere.

This manual is for everyone who touches the app:

- **Field capture** - whoever records animals and events out at the herd, usually from a phone or tablet.
- **Farm admin / office staff** - whoever manages the master herd list, corrects records, and runs the server itself.

Table of Contents

1. [Overview & Concepts](#)
 2. [Server Setup](#)
 3. [Device Setup](#)
 4. [Field App - Recording the Herd](#)
 5. [Admin - Dashboard](#)
 6. [Admin - Herd](#)
 7. [Admin - Settings](#)
 8. [Troubleshooting / FAQ - Annexe A: Data Field Reference](#)
-

1. Overview & Concepts

What Kudde is (and isn't)

Kudde is deliberately small. There is a proper international standard for recording livestock data - ICAR's Animal Data Exchange - and it defines around 75 resource and event types: breeding values, embryo flushing, carcass ultrasound, DNA results, formal conformation scoring, and more, built for stud and champion breeding operations. A small-scale farm needs a fraction of that: who an animal is, and the handful of things that happen to it. Kudde keeps the identity fields a stud registry would recognise (ear tag, breed, sex, parentage, birth date) and the event types that actually apply on a small farm (birth, weight, treatment, movement, death, sale) - trimmed down, not guessed from scratch.

There's no herdbook registration, no breed-code lookup, no formal scoring. Breed is free text. Parentage is recorded by tag, not by looking an animal up in this database - a sire or dam may have been sold, may have died, or may never have been entered here at all (a bought-in animal, a bull borrowed from a neighbour), so it has to be recordable even when it points at nothing Kudde knows about.

The two device roles

Every device opens one of two fixed web addresses, depending on what it's for:

Role	Address	Used by	What it does
Field	/field/	Whoever is out with the herd	Add animals, record events, move a camp, take photos, browse the herd - works offline
Admin	/admin/	Farm office	Full herd list with editing, dashboard, complete event history, manual event entry, farm settings

Unlike some setups, there's no one-time device-ID screen that decides this for you - a device is whichever screen you open on it, and it's fine to have several tabs or devices open on either screen at once.

The network is the password

There is still no sign-in and no password. What replaced it is the route a request arrives on: **Kudde answers over Tailscale and nothing else - Field included.**

A phone on the farm's own Wi-Fi gets the same refusal an outsider would. So does a browser on the server PC's own console, reached over AnyDesk or sitting in front of it - `http://localhost:8030/` is refused exactly like anywhere else. That last part is deliberate rather than an oversight: a loopback

exemption is one that any remote-desktop session into that PC would inherit for free, and remote desktop is exactly how a farm server is normally reached.

The one thing that answers off Tailscale is `/healthz`, which reports only that the process is up and says nothing about the herd. The installer's own post-install check depends on it, because it runs before Tailscale is necessarily configured.

Everything you open therefore looks like:

```
https://<pc-name>.<your-tailnet>.ts.net:8030/field/  
https://<pc-name>.<your-tailnet>.ts.net:8030/admin/
```

The port matters. See [Publishing Kudde over Tailscale](#) in chapter 2 for how to set this up, and why `:8030` is not optional on a PC that also runs Boord.

If you open the wrong address, the server answers in words rather than failing silently:

Kudde is only reachable over Tailscale - open the secure `https://...ts.net:8030/` address (the port matters on a PC that also runs Boord), not the farm wifi one and not localhost

Anyone on your tailnet who reaches `/admin/` can still edit or delete herd records - there are no per-person permissions. The tailnet is the boundary, so keep it to devices and people you'd hand the herd book to.

Why field capture is offline-first

Cattle aren't always where the WiFi is. The Field app is built so that **recording an animal or an event never depends on having a connection** - it's saved on the device first, and quietly synced to the server in the background whenever a connection becomes available. You can keep working through a signal dropout without losing anything or needing to notice it happened.

Animal status, and what changes it

Every animal is **alive**, **dead**, or **sold**. Nobody has to remember to update this separately: recording a **death** or **sale** event against an animal moves it to that status automatically, the moment the event is saved - in the Field app or in Admin, online or (for Field) queued offline. There's no way to record a death or sale event without the animal's status following it.

The six event kinds

Kind	What it records	Extra fields used
Birth	The animal was born	-
Weight	A weighing	Value, in kg

Kind	What it records	Extra fields used
Treatment	A vet treatment, dosing, etc.	-
Movement	A move between camps/paddocks	Location (the camp name)
Sale	The animal was sold	-
Death	The animal died	-

Every event also carries a date and a free-text note, whatever the kind. `value` (kg) only means anything on a weight event, and `location` only means anything on a movement event - the other kinds simply leave those blank.

A weight event must carry a weight. Both apps and the server refuse a weighing with the kg box left empty, or filled in with something that isn't a number. This is stricter than it looks worth being: the "needs weighing" list is driven by the *date* of the most recent weight event, so a blank one would silence the reminder for another six months while recording nothing - the animal then becomes invisible to the one screen meant to catch it.

Weather, recorded for you

Once the farm's position is set in [Admin - Settings](#), every event recorded from then on is stamped with that day's weather at the farm - highest and lowest temperature, and rainfall in mm - looked up automatically from [Open-Meteo](#). Nobody types it in.

It is best-effort by design. If the farm has no position set, or the server has no internet at that moment, the event is still recorded and simply carries no weather. **A weather lookup never blocks a record from being saved.** Events recorded before you set the position keep no weather; it is not backfilled.

The Field app's quick-action buttons cover weight, treatment, movement, sale, and death - the events you'd realistically capture while standing next to the animal. There's no field quick-action for **birth**, since a newly recorded animal's birth date is normally set directly on the **Add animal** form instead. A birth event is there for the rarer case where you want to log the birth itself as a dated event with its own note - it is available from **Admin - Herd's** event form.

Camps, and moving a whole one

Kudde has no separate list of camps to set up and maintain. A camp is just the location typed on a movement event, and an animal's current camp is worked out from its history - the most recent movement (or birth) that named one.

Because a camp move in real life is a whole group walking through a gate together, both apps have a **Move camp** action that records the same move against every animal in a camp at once, rather than one tag at a time. See [Moving a camp](#) in chapter 4.

Photos

An animal can carry photos - taken with the phone's camera out at the herd, or picked from a file in the office. They're stored as ordinary image files on the server, beside the database, and shrunk on arrival so a farm PC's disk isn't filled by a few hundred camera originals. See [Taking photos](#) in chapter 4.

"Needs weighing"

An alive animal with no weight event recorded in the last **182 days** (roughly six months) shows up as needing weighing, on both the Field app's summary and the Admin dashboard. Long enough that weighing every muster or two doesn't nag you; short enough to actually catch an animal that's fallen through the cracks. This is purely informational - nothing stops you from ignoring it - but it's the one thing in Kudde that proactively tells you an animal hasn't been looked at in a while.

2. Server Setup

Kudde is a single Python service (FastAPI) that serves its own frontend and stores everything in a local SQLite database file - there's no separate database server, cloud account, or build step. It runs on one ordinary computer (the "server"), and every other device just opens a web address pointing at that computer - needing only a browser and the Tailscale app, which is what gets it onto the farm's network in the first place.

Prerequisites

- A Windows, Mac, or Linux computer that stays switched on and connected to the network while the app is in use. It doesn't need to be powerful - it's one lightweight Python process and a small SQLite database, not a heavy computation.
- Python 3.9 or newer (the automated Windows installer gets you 3.11 if nothing suitable is already present).
- **A Tailscale account, and Tailscale installed on the server PC and on every phone, tablet or PC that needs Field or Admin**, all signed into the same tailnet. This is not optional any more - it is how Kudde decides who may reach it at all. It's free for a farm-sized number of devices; get it from tailscale.com/download.
- The project folder, copied onto that computer. Prefer `git clone` over a zip or USB copy: the signed-update mechanism in [Updating to a newer version](#) needs a real clone to work from.

Don't put the project folder inside a synced cloud drive - Google Drive, OneDrive, Dropbox, iCloud. The sync client keeps file handles open, which can block updates or stop a folder from deleting, and syncing a live SQLite database file while it's being written is a known way to corrupt it. Back a finished backup copy up to the cloud by all means; just don't run the app from a synced folder.

Quick setup on Windows: the automated installer

The project folder includes `install.bat`, which does everything described below in one step and is safe to double-click again later if something needs redoing.

1. Get the project folder onto the PC.
2. Double-click `install.bat`.
3. If Windows shows a blue **"Windows protected your PC"** screen, click **"More info"**, then **"Run anyway"** - normal for any script that isn't from a large, registered publisher.
4. If a **User Account Control** prompt appears, click **"Yes"** - administrator rights are needed to configure the firewall and register the auto-start task.

5. Wait for it to finish. It will: - Install Python 3.11 if no compatible 64-bit Python 3.9+ is already on the PC.

- Create a virtual environment in `backend\.venv` and install the app's dependencies into it.
- Write a `start_server.bat` launcher.
- Open port **8030** through Windows Firewall. Boord runs as 8000 and 8020 on a real farm server; Kudde nearly collided with Boord Owner over 8010 on exactly that kind of shared PC, so 8030 continues the sequence rather than reusing a number already spoken for.
- Register a Scheduled Task named exactly "**Kudde Server**" that starts the server automatically at boot, running as SYSTEM - no one needs to be logged in for it to start.
- Start the server immediately and poll `http://localhost:8030/healthz` for up to 20 seconds to confirm it actually came up. It checks `/healthz` rather than the app itself because the app refuses anything that didn't arrive over Tailscale, which this PC's own console hasn't yet at install time.
- Print the Tailscale commands you still need to run, and the fingerprint prompt for the release key.

1. **The installer cannot finish the job on its own.** Two things are left for you, both described below: publishing the port over Tailscale, and telling the server which release key to trust.

Publishing Kudde over Tailscale

Do this **once**, on the server PC, after Tailscale is installed and signed in there:

```
tailscale serve --bg --https=8030 http://localhost:8030
```

That publishes Kudde on this PC's Tailscale name, on port 8030, over HTTPS. Then find the PC's name:

```
tailscale status
```

and open `https://<pc-name>.<your-tailnet>.ts.net:8030/field/` (or `/admin/`) from any device signed into the same tailnet - including the server PC itself, which gets no exemption.

On a PC that also runs Boord, the `--https=8030` part is not optional. Boord already answers on the bare `https://...ts.net/` slot with no port. Running `tailscale serve` in its bare form for Kudde silently *takes that slot over* and breaks Boord, rather than erroring. The `--https=8030` form is Kudde's own slot and cannot collide. Run `tailscale serve status` to see what is already claimed.

Trusting the release key

The other thing the installer leaves to you. Kudde only installs updates signed by a key you have told it to trust, and the fingerprint has to be typed in by a person from a source they trust - not read out of the repository, which is the very thing it's there to check.

The fingerprint is stored in `data\release_key.fpr`, deliberately outside the code, so an update can't rewrite the thing vouching for it. The full procedure is in `UPDATING.md`, under "Trusting the release key".

Setting up by hand (Windows, Mac, or Linux)

Useful if the installer fails partway, or on Mac/Linux where there's no automated script yet. From the `backend/` folder:

Windows:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8030
```

Mac/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8030
```

- `--host 0.0.0.0` lets `tailscale serve` reach the process. It does **not** open the herd to the farm Wi-Fi: a request that didn't arrive over Tailscale is refused by the app itself, whatever address it came to.
- `8030` is the port every other step in this manual assumes, and the one `tailscale serve` is pointed at above. Any free port works, but change it in both places and stick with one number.
- On Windows, allow the port through Windows Firewall if it isn't prompted automatically (Windows Security → Firewall & network protection → Advanced settings → Inbound Rules → New Rule → Port → TCP 8030 → Allow).
- To keep it running after closing the terminal and restart automatically after a reboot, use your platform's standard approach for a long-running service (a Scheduled Task on Windows - see below - a `launchd` agent on Mac, or a `systemd` service on Linux) pointed at the same `uvicorn` command.

What happens on first startup

The very first time the server runs, it creates `data\kudde.db` and brings the database schema up to date automatically (via Alembic migrations, run from `main.py`'s startup step) - there is nothing to run by hand for this, on a fresh install or after pulling an update that changes the schema. **No demo or sample data is seeded** - the herd list starts empty, ready for real animals.

On every later start, if the schema needs to move, the server **copies the database first and refuses to migrate if that copy fails** - see [Backing up your data](#). "The server didn't start" is recoverable in a way "the migration half-finished" is not.

Updating to a newer version of the code

A farm server installs **signed releases**, not whatever is on `main`. Pushing code deploys nothing on its own: the server checks out a tag whose signature matches the release key it was told to trust, and refuses anything else. That check is the whole point - a bad or tampered update on a farm means somebody's herd records.

To install an update, on the server PC:

- Double-click `update_server.bat`. It stops the server properly, verifies the newest signed tag, copies the database and photos, migrates, and restarts.

To be told when a release is out, double-click `setup_update_check.bat` once. It registers a daily check that writes down what it found; Admin then shows a notice at the top of the Dashboard when a newer signed release is available, and a different, red one if the check itself has been failing - a check that has been broken since April otherwise looks exactly like "no updates".

`UPDATING.md` in the project folder is the full reference: what the updater does step by step, how to do it by hand, what to do if an update goes wrong, and how releases are cut and signed in the first place.

Each device's browser tab or installed app also needs a full close-and-reopen after an update to pick up new frontend code - see [Confirming a device picked up an update](#) in chapter 3.

Stopping, starting, and restarting the server

If set up via `install.bat`, the server runs as a Scheduled Task named "**Kudde Server**", as SYSTEM, with no visible window.

Using Task Scheduler: open **Task Scheduler**, find "**Kudde Server**" in the Library, right-click → **End** to stop it, or **Run** to start it. To restart, do **End** then **Run**.

Using Command Prompt (as administrator):

```
schtasks /end /tn "Kudde Server"
schtasks /run /tn "Kudde Server"
```

Simplest restart: since the task launches automatically on every startup anyway, just restarting the PC has the same effect.

If you need it definitely stopped - before copying the database, for example - use `stop_server.ps1` rather than End on its own:

```
powershell -NoProfile -ExecutionPolicy Bypass -File stop_server.ps1
```

Ending the Scheduled Task stops the launcher, but the server process it spawned is separate and can survive that, still holding port 8030 and the database open. This script ends the task, then waits until port 8030 is actually free, and reports what's still holding it if it never comes free. It only ever stops processes running from Kudde's own `backend\.venv` - if something else on the PC has taken port 8030, it says so and refuses to kill it.

Checking it's running: browse to `http://localhost:8030/healthz` on the server PC - if it answers `{"status": "ok"}`, the process is up. Don't check `/field/` here: that is refused from the console like anywhere else, so a healthy server and a broken one look identical.

Backing up your data

Everything Kudde knows lives in the `data\` folder, in two parts:

- `data\kudde.db` - the herd: every animal, event, photo record and the farm's own details.
- `data\photos\` - the photo files themselves. The database stores only their filenames, so one half without the other is not a backup.

What Kudde does for you. Immediately before any update that changes the database schema, the server takes a rollback copy into `data\backups\` : a consistent copy of the database, plus the photos that belong with it, keeping the five most recent. If that copy can't be written, **the migration is refused and the server doesn't start** - deliberately, because nothing has changed at that point and checking the previous release back out is a complete fix. The photos are taken as hard links, so they cost names rather than disk space, and a failure to take them is only warned about: a migration rewrites the database, never a photo file.

What is still on you. Those copies are on the same PC and the same disk. They protect you from a bad update, not from a dead drive, a fire, or a theft. For a real backup, stop the server (see above) so nothing is mid-write, then copy the whole `data\` folder somewhere off the machine - a USB drive or a cloud drive. Copying a finished folder into a sync folder is fine, even though running the app from one isn't. Do this regularly; a pre-migration copy is not a substitute.

Removing Kudde from a PC

Double-click `uninstall.bat` . It stops and unregisters the auto-start Scheduled Task, removes the firewall rule, deletes the generated `start_server.bat` , and removes the Python virtual environment. It prints what's still on the machine afterward.

It does not delete data\ , and it does not delete the project folder. That folder holds the herd database, the photos, and the pre-migration copies - every animal and event recorded - and reinstalling doesn't bring any of it back. Deleting it stays a deliberate act, so the uninstaller prints the commands rather than running them:

```
xcopy /E /I "C:\Kudde\data" "%USERPROFILE%\Desktop\kudde-data-backup"  
rmdir /s /q "C:\Kudde"
```

Copy first, and run the second command from a folder outside C:\Kudde - a Command Prompt sitting inside it will block its own delete.

3. Device Setup

There's no device-registration step. A device is whichever address you open on it:

- **Field:** `https://<pc-name>.<your-tailnet>.ts.net:8030/field/`
- **Admin:** `https://<pc-name>.<your-tailnet>.ts.net:8030/admin/`

Run `tailscale status` on the server PC to find `<pc-name>`. The device you're setting up has to be signed into the same tailnet, or neither address answers - see [The network is the password](#).

Bookmark the right one on each device, or better, install it as an app icon.

Installing as an app icon (optional, recommended)

Both Field and Admin are small installable apps (PWAs) with their own name and navy icon, so a device can show "Kudde Field" or "Kudde Admin" on its home screen instead of a browser tab or bookmark. This isn't required - the browser bookmark works fine on its own - but it makes the right screen one tap away.

- **Android (Chrome):** tap the three-dot menu → **"Add to Home screen"** (or use the install banner/icon in the address bar if offered automatically).
- **iPhone/iPad (Safari):** tap the **Share** icon → **"Add to Home Screen"**.
- **Windows/Mac (Chrome or Edge):** look for an install icon (a monitor with a down-arrow) at the right end of the address bar, or the three-dot menu → **"Install [app name]..."**.

Install from the address you actually intend to use (`.../field/` or `.../admin/`) so the shortcut opens the right screen with the right name and icon.

Confirming a device picked up an update

Each screen's header shows its version number (e.g. `v0.3`) next to the sync status pill. Admin goes further: it asks the server what version *it* is running, and if the two disagree it shows both, because that disagreement is the signal - either this screen is stale, or the code was checked out without restarting the server. Because the app is a PWA with a service worker, an installed icon can keep showing the old frontend after the server itself has been updated, until it's fully closed and reopened. If a device looks like it hasn't picked up a recent change, check the version number first; if it's behind, close the app completely (not just switch away from it) and reopen it. A leftover home-screen icon from an old install can also point at stale cached data - if renaming or re-icon-ing an install, remove the old icon and clear the site's data before reinstalling.

4. Field App - Recording the Herd

Header

Shows "Kudde / Field", the sync status pill, and the app version. The sync pill reads:

Pill text	Meaning
Online	Connected, nothing waiting to sync
Syncing N...	Connected, N queued items being sent now
Offline	Can't reach the server right now
Offline - N pending	Can't reach the server; N items are queued and will send once it's back

The offline banner

A banner reading *"Offline - changes are saved on this device and will sync later"* appears whenever the app can't reach the server. It's informational, not a block - every screen still works.

Pull down to refresh

Installed as a home-screen app, there's no browser reload button, so pull down from the top of the list to re-fetch the herd and dashboard summary (and trigger a sync attempt) by hand. This is a touch gesture only.

Searching and adding an animal

The search box at the top filters the herd list live, by tag (matches anywhere) or name. **Add animal** opens a form for:

- **Tag number** (required) - the ear tag number, and the animal's permanent identity. Kudde refuses a tag already in use, and refuses a blank one. Surrounding spaces are trimmed off before it's stored, so " A1 " and "A1" are the same animal rather than two identical-looking ones nothing can tell apart.
- **Name** (optional)
- **Sex** (required) - Female or Male
- **Breed** (optional, free text)
- **Birth date** (optional)

- **Sire tag / Dam tag** (optional) - the parent's ear tag, if known. These don't have to exist in Kudde's own records; a bought-in animal or a bull borrowed from a neighbour can still be recorded as a parent.

New animals start **Alive**.

The herd summary and list

Below the search box, two counts: **Alive**, and **Need weighing** (see "[Needs weighing](#)" in chapter 1). The list below shows every cached animal, sorted by tag, each with a status pill (Alive/Dead/ Sold). Tap any animal to open its detail.

Animal detail and recording an event

Tapping an animal opens its tag, name/breed/sex, status pill, and five quick-action buttons:

Weight · Treatment · Move · Sale · Death

Each opens a small form: a date (defaults to today), a value in kg (weight only), a camp/location (movement only), and a free-text note. Saving:

- Sends the event to the server immediately if reachable.
- If not, saves it to this device's outbox and shows *"Saved on this device - will sync when online"* instead.
- For a **Sale** or **Death** event, flips the animal's status right away on this device (the same rule the server applies), so the list and the detail card are correct immediately, connected or not.

A **Weight** event with the kg box empty is refused rather than saved blank - see [The six event kinds](#).

Taking photos

The detail screen has a **Take photo** button. On a phone it opens the camera directly; on a PC it opens a file picker. Photos appear as a row of thumbnails - tap one to open it full size, or the small ✕ on its corner to delete it.

Photos work offline the same way events do: taken with no signal, the photo is held on the device and uploaded when a connection comes back. The thumbnails themselves are fetched live, so the row reads *"Offline - photos unavailable"* until you're reconnected.

The server shrinks every photo when it arrives - to 1600 pixels on the long edge, re-encoded as JPEG - so a few hundred camera originals don't fill the farm PC's disk. It also turns a sideways phone photo the right way up permanently, rather than relying on a tag that the re-encode would throw away. Uploads above 15MB are refused.

Below the action buttons, **History** lists every recorded event for that animal, newest first - unavailable while offline, since it isn't cached locally the way the herd list is.

Moving a camp

Move camp, on the main screen, records one move against a whole group rather than one animal at a time:

1. It lists every camp that currently holds at least one alive animal, with how many are in each. (A camp only appears once something has been recorded as being there - see [Camps](#) in chapter 1.)
2. Pick the camp everyone is moving *from*. It shows that camp's animals, all ticked.
3. Untick any that aren't going.
4. Fill in the date, the new location, and an optional note, then **Move**.

That writes one movement event per animal, all with the same date, location and note. Like everything else in Field it queues offline, and syncs as a single item rather than one per animal.

Offline-first capture, in detail

Adding an animal or recording an event always succeeds locally first. Behind the scenes:

- The herd list is cached on the device (IndexedDB), so it's browsable offline - though each animal's event *history* is fetched live and isn't available offline.
 - Anything that couldn't reach the server goes into an on-device outbox and is retried automatically: every 20 seconds, the moment the browser reports it's back online, and on every pull-to-refresh. The outbox carries new animals, events, whole camp moves, and photos alike.
 - **A retry can't record the same thing twice.** Each capture is stamped with an id on the device at the moment it's taken, and the server stores one record per id. This matters more than it sounds: the app gives up on a request after 8 seconds and re-queues it, so a request that was slow but actually landed *will* be sent again - and without that id, a slow connection would quietly add a second weighing, a second camp move for every animal in the camp, or a second copy of a photo. Two genuine captures are still two records; the same animal really can be treated twice in one day.
 - If the server later rejects a queued item outright (most commonly, a tag that got created from another device first and synced sooner), that one entry is dropped rather than blocking every entry queued behind it. **Two devices creating the same new tag while both offline is the one case Kudde can't reconcile for you** - whichever syncs first wins, and the second is silently lost. If more than one device might be adding new animals, agree tag ranges in advance, or check the Herd tab in Admin before working somewhere with no signal.
-

5. Admin - Dashboard

The Admin app's landing tab. If the server can't be reached, the sync pill in the header reads **"Offline - the server on this network can't be reached"** and the tab shows whatever it last loaded - there's no offline cache here the way the Field app has one, since Admin is meant to be run from the office, on a machine that can reach the server.

Update notice

When a newer signed release is available, a notice appears at the top of the Dashboard naming it and telling you to run `update_server.bat` on the server PC. A red version of the same notice means the update *check* couldn't verify the newest release - nothing has been installed, and `UPDATING.md` is the place to start. Neither appears until `setup_update_check.bat` has been run once on the server; see [Updating](#) in chapter 2.

Herd counts

Three tiles: **Alive**, **Dead**, **Sold** - a live count straight from the database.

The needs-weighting list

Every alive animal with no weight event in the last 182 days (see [chapter 1](#)), with a **View** link that jumps to the Herd tab and opens that animal's edit screen directly.

Recent activity

The 10 most recent events across the whole herd, newest first - whatever kind, from whichever device recorded them. Each shows its date and, once the farm's position is set, that day's weather beneath it (for example `18-27°C, 4mm`). Events with no weather recorded simply show the date.

6. Admin - Herd

The full, editable herd list.

Searching and filtering

A search box (tag or name) and a status dropdown (All / Alive / Dead / Sold) filter the table live. **Add animal** opens the same fields as the Field app's Add animal form, and **Move camp** runs the same whole-camp move described in [chapter 4](#) - the office version of the same action, for a move somebody phoned in rather than captured at the gate.

The herd table

Columns: Tag, Name, Breed, Sex, Birth date, Status. Click any row to open that animal.

Editing an animal

Opens every field the Add form has, plus **Status** (Alive/Dead/Sold) - editable directly here, in addition to being set automatically by a sale or death event. The **tag number can't be changed** once an animal is created; it's the identity everything else (events, parentage links from other animals) is recorded against.

Photos on the animal record

The same photo strip as the Field app, on the animal's edit screen: **Add photo** opens a file picker (there's no camera capture here, since Admin is an office screen), thumbnails open full size on click, and the ✕ on a thumbnail deletes it. Deleting removes the file from the server, not just the listing.

History and recording an event

Below the fields, when editing an existing animal: its full event history, and a compact form to record a new one directly - date, kind (**Weight, Treatment, Movement, Birth, Sale, Death** - including Birth, unlike the Field app's quick actions), value in kg (weight only), location (movement only), and a note. Each event in the history shows its weather alongside the date, where one was recorded. Saving refreshes the history list, the herd table, the dashboard, and - if the event was a sale or death - the status pill shown at the top of this same modal, without needing to reopen it.

Admin has no outbox: it's meant to be run from a machine that can reach the server. A save that can't reach it says so rather than queueing.

7. Admin - Settings

The farm's own details, and the one setting that changes what gets recorded. Admin only - there's no Settings screen in Field.

Farm details

Farm name, **Farmer name** and **Phone number**, all optional and all free text. Kudde doesn't act on any of them; they're there so a farm's own identity lives with its records rather than only in the head of whoever set the server up.

GPS coordinates, and the weather

Latitude and **Longitude** - the farm's single position. Set them in any of three ways:

- Click anywhere on the map, or drag the pin.
- Press **Use current location** (the browser will ask permission - on a desktop PC this can be some distance out, so check the pin afterwards).
- Type the numbers in directly.

Press **Save settings** to store them. Nothing is saved until you do.

This is what [weather recording](#) keys off. Once a position is set, every event recorded from then on carries that day's weather at the farm. It is one position for the whole farm, not one per camp - a camp move records the weather at the farm, not at either camp.

Clearing the coordinates stops new events carrying weather. It doesn't remove weather already recorded against past events.

The map tiles come from OpenStreetMap over the internet. On a farm server with no connection the rest of the screen still works - you'll get an empty grey map and can type the coordinates in by hand.

8. Troubleshooting / FAQ

The page says "Kudde is only reachable over Tailscale". The request didn't arrive over Tailscale. Nearly always one of: you opened the farm Wi-Fi address or `localhost` instead of the `.ts.net` one; this device isn't signed into the tailnet (check the Tailscale app on it); or `tailscale serve` was never run on the server PC, or was run without `--https=8030`. Run `tailscale serve status` on the server to see what it's actually publishing. This message is the server working correctly, not a fault.

Admin says "Offline - the server on this network can't be reached". The server PC is off, asleep, or off the tailnet. Confirm the PC is on and the process is up (`http://localhost:8030/healthz` answers `{"status": "ok"}` on the server PC itself), that `tailscale status` lists it, and that this device is on the same tailnet.

Field says "Offline - N pending". Normal while out of signal range - those N animals/events are saved on this device and will send automatically once it reconnects (or pull down to refresh to retry immediately). Nothing is lost by staying offline; it's only lost if the device itself is lost or its browser data is cleared before it syncs.

"Tag ... is already in use". Tags are permanent identities - even a dead or sold animal keeps its tag forever, so it can't be reused for a new one. Search for it in Admin → Herd with the status filter cleared to find the existing record.

Two devices added the same new tag while both were offline. See [Offline-first capture, in detail](#) - this is a known limitation, not a bug: whichever syncs first keeps the tag, and the other capture is dropped. Agree on tag ranges up front if more than one device might be adding new animals away from a connection.

A device looks like it hasn't picked up a recent update. Check the version number in the header first (see [chapter 3](#)); if it's behind, fully close and reopen the app rather than just switching away and back. In Admin, a header showing two different version numbers means the screen and the server disagree - the screen is stale, or the code was checked out without restarting the server.

The server won't start, or a browser says the connection was refused. Something may already be listening on port 8030 - most often a leftover `python.exe` from a manual `uvicorn` run left in Task Manager after a Scheduled Task restart. End it, then start the "Kudde Server" task again, or use `stop_server.ps1` first to confirm the port is genuinely free.

The "Kudde Server" scheduled task shows as ran/ready, but nothing answers. Check the task's History tab in Task Scheduler for an error, or run `start_server.bat` by hand in a visible Command Prompt window - errors that a headless Scheduled Task swallows are printed there.

How do I get my data off this PC, or back it up? Everything is in the `data\` folder - the database *and* the photos, which are separate files. Copy the whole folder. See [Backing up your data](#) in chapter 2: Kudde takes its own copy before an update, but that copy sits on the same disk, so an off-machine backup is still a habit worth keeping.

Can I reach Kudde from off the farm? Yes - that's now the only way anyone reaches it. Any device signed into the same tailnet opens the same `.ts.net` address from anywhere with a connection; there's nothing extra to set up per location. What you must **never** do is forward port 8030 through your router to the public internet. Kudde has no sign-in, and the tailnet is the only thing standing between your herd records and whoever finds the address.

A photo won't upload / a camp move seems stuck. Both queue on the device like any other capture and retry automatically; the sync pill counts them. A photo is several MB, so on a weak connection it can take a few attempts - it won't be recorded twice as a result, see [Offline-first capture](#).

An event has no weather on it. Either the farm's position wasn't set when it was recorded, or the server couldn't reach the internet at that moment. Both are by design: the event is recorded either way, and weather is never backfilled onto past events. Set the position in [Settings](#) and new events will carry it.

The map in Settings is blank. Its tiles come from OpenStreetMap over the internet, so a server with no connection shows an empty grid. Type the coordinates in by hand instead - the rest of the screen still works.

The server refused to start after an update, saying it couldn't back up the database. That's deliberate, and nothing has changed - the previous release still runs against your data. It's almost always a full disk; free some space in `data\backups\` and start the server again. See [Backing up your data](#).

Annexe A: Data Field Reference

Animal

Field	Type	Notes
id	integer	Internal id, assigned by the database
tag	text	The ear tag number. Unique, and the animal's permanent identity
name	text	Optional
breed	text	Optional, free text - no breed-code lookup
sex	male female	Required
birth_date	date	Optional
sire_tag	text	Optional. The father's ear tag - not required to exist as an Animal in this database
dam_tag	text	Optional. The mother's ear tag - same rule as sire_tag
status	alive dead sold	Set to alive on creation; changes automatically when a death or sale event is recorded, or can be set directly in Admin
created_at	timestamp	Set once, on creation
updated_at	timestamp	Updated on every change

Event

Field	Type	Notes
id	integer	Internal id
animal_id	integer	The animal this event belongs to
kind	birth weight treatment movement death sale	
event_date	date	Required
value	number	Weight in kg - only meaningful on a weight event

Field	Type	Notes
note	text	Optional, free text, any kind
location	text	The camp/paddock name - only meaningful on a <code>movement</code> event. An animal's current camp is the most recent <code>movement</code> or <code>birth</code> event that named one
weather_temp_max	number	°C at the farm on <code>event_date</code> , filled in automatically. Null if the farm has no position set, or the lookup failed
weather_temp_min	number	°C, same rule
weather_precipitation	number	mm of rain, same rule
client_uuid	text	The id the capturing device gave this event. Unique. Null for anything captured in Admin, which has no outbox. What makes a replayed capture record once
created_at	timestamp	Set once, on creation

`death` and `sale` events additionally set the parent `Animal.status` to `dead` or `sold` respectively, the moment they're recorded.

A weight event is refused unless `value` is a number greater than zero.

AnimalPhoto

Field	Type	Notes
id	integer	Internal id
animal_id	integer	The animal this photo belongs to
filename	text	The file's name under <code>data/photos/<animal_id>/</code> . A fresh uuid, not the phone's original filename, so two photos can never collide
content_type	text	Always <code>image/jpeg</code> - everything is re-encoded on arrival
caption	text	Optional, free text
client_uuid	text	Same contract as <code>Event.client_uuid</code> : unique, null from Admin, and what stops a re-sent upload storing a second copy
created_at	timestamp	Set once, on creation

The image bytes are never stored in the database - only the filename. The files live beside it in `data/photos/` , which is why a backup has to take both.

Farm

One row per install, holding the farm's own details. Created the first time [Settings](#) is opened.

Field	Type	Notes
id	integer	Always 1 - there is one farm per server
farm_name	text	Optional, free text
farmer_name	text	Optional, free text
phone_number	text	Optional, free text
gps_lat	number	The farm's latitude. What each event's weather is looked up against
gps_lng	number	The farm's longitude
updated_at	timestamp	Updated on every save